Keep Learning or Start Over?

Plasticity-Preserving Updates for Bearing Vibration Forecasting

Elissa Ephrem eee19@mail.aub.edu
Galia Akl gra16@mail.aub.edu
Karim Bteish krb05@mail.aub.edu

Project proposal | 20 October 2026

1. Problem and motivation

A model that monitors a machine has to keep learning, because the machine changes as it wears. But a neural network that is trained for a long time can slowly lose its ability to learn new patterns. This is called *loss of plasticity* [1].

Question. When a bearing degrades, should a vibration forecaster keep updating its weights, refresh part of them, or start over? We compare simple "plasticity-preserving" updates with ordinary continual training and with periodic retraining from scratch, at a similar computing cost. A secondary question is whether signals inside the network can tell us *in advance* when a refresh will help.

Task. We predict the vibration level one hour ahead: the logarithm of the RMS vibration amplitude, $y_t = \log_{10} \mathrm{RMS}(t + 1\,\mathrm{h})$, from current and past vibration features. This is a regression task. It is not a failure alarm or a remaining-life estimate.

Measurable goal. A plasticity-preserving update succeeds if it has a lower mean absolute error (MAE) than both ordinary updating and periodic retraining on the held-out run. We also compare against simple "persistence" forecasts and check errors near the end of each bearing's life. We do not assume that any method will win.

2. Impact

The practical decision is whether a monitoring model should keep its weights, refresh some of them, or restart. A positive result would show maintenance teams which option gives the best accuracy for the computation spent. A negative result is also useful: it would show that resets can destroy useful knowledge, or that easier training does not always give better forecasts.

The most harmful error is *underprediction*, which means forecasting less vibration than actually occurs while a bearing is failing. We therefore report it separately. Our data come from one laboratory rig with few failures, so the results cannot prove industrial safety. No maintenance decision will be automated.

3. Related work

Warm starts. Ash and Adams [2] showed that a network trained further from an old solution can generalise worse than one trained from scratch, even with the same training loss. They proposed *shrink-and-perturb*: shrink the weights and add small random noise. We test this method on a forecasting task that arrives over time.

Loss of plasticity. Dohare et al. [1] showed plasticity loss in long image and reinforcement-learning experiments. Their fix, continual backpropagation, reinitialises rarely used neurons. Their work motivates our fair, equal-budget comparisons. We do not assume the effect will appear on bearing data.

Inactive neurons. Sokar et al. [3] linked a growing number of "dormant" (almost always inactive) neurons to lost capacity, and proposed ReDo, which resets them. We test a ReDo-style reset on regression.

Interpreting internal signals. Lyle et al. [4] found that plasticity loss can happen even without inactive neurons, so dormant neurons alone do not prove the mechanism. A recent preprint [5] showed that some common internal measures fail to predict whether a network can still learn. We therefore test internal signals against a simple alternative: recent forecast error.

Closest bearing work. Yang et al. [6] add selective re-initialisation to a deep model for remaining-life prediction on other bearing datasets, and report lower errors. To our knowledge, they do not compare against periodic retraining at an equal budget, or test whether internal signals predict when a reset helps. Those two comparisons are our contribution. We do not claim a new reset method.

4. Data

Source. We use the IMS Bearing Data Set, collected by the University of Cincinnati's Center for Intelligent Maintenance Systems with Rexnord and distributed by NASA [7, 9] (https://data.nasa.gov/dataset/ims-bearings). NASA asks users to acknowledge the source; no other licence terms apply to our use.

Four bearings on one shaft were run until some of them failed, in three separate runs (Tests 1–3). Every 10 minutes, an accelerometer recorded a one-second vibration snapshot of each bearing. This gives 12 bearing histories, but only three runs and four failures. Bearings on the same shaft are not independent.

Processed features. We start from feature tables published by a public repository that processed the original recordings [8]. We confirmed that our files are identical to that repository's. In week 1 we will recompute the

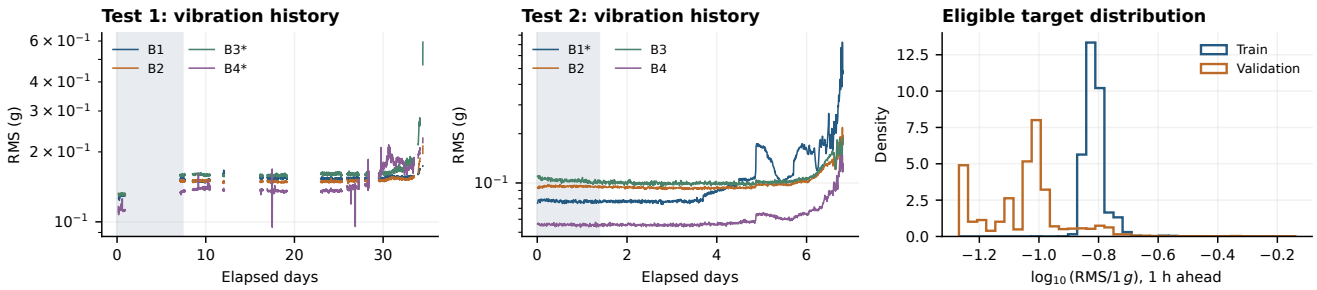


Figure 1: Tests 1 and 2. Left: vibration level (RMS) of each bearing over time; * marks a bearing that failed, grey marks the calibration period, and gaps are breaks in recording. Right: distribution of the forecast targets. Most values sit in a narrow band; the tail comes from degrading bearings. Data: IMS feature tables [7, 8].

Table 1: Development data after preprocessing (all four bearings).

	Test 1	Test 2
Snapshots per bearing	2,156	984
Usable forecast pairs	6,428	3,104
Recording gaps	24	0
Missing values	0	0

Table 2: Baselines already run: MAE of the log vibration level (lower is better). "Late" covers the final 10% of the life of failed bearings.

Model	Train	Validation	Late
Persistence (last value)	0.0038	0.0113	0.0694
Persistence (average)	0.0034	0.0112	0.0770
Ridge regression	0.0034	0.0112	0.0759
Neural network	0.0034	0.0111	0.0762

features ourselves from the original NASA recordings and compare them; our extraction script is ready.

Inputs and preprocessing. Each snapshot is summarised by 20 features: vibration amplitude, measures of signal shape, and the energy at the frequencies where specific bearing faults appear. We add a short moving average of each feature, giving 40 inputs. The first part of each bearing's history is used only to set the scaling of each feature, and is not used for training or scoring. We only form forecast pairs within continuous recording, so gaps are never filled in. We use one accelerometer per bearing.

Limitations. The data have recording gaps (Table 1), measurement noise, one operating condition and few failures. A bearing that did not fail is not a certified healthy control.

5. Models and preliminary results

Simple baselines. *Persistence* forecasts that the vibration level in one hour equals the latest value, or the average of the last six. *Ridge regression* [10] predicts the change from that average using the 40 features. It tests whether the features add any information. We train on Test 1 and select settings on Test 2.

Neural network. We implemented a small two-hidden-layer network from scratch in NumPy [11], including backpropagation and the Adam optimiser [12], and checked the gradients numerically. It predicts the same change as ridge regression.

What the baselines show. Ridge and the network are only slightly better than persistence overall, and the simplest persistence forecast is best near failure (Table 2). The network's validation error was lowest after its first epoch: training longer only fitted Test 1 more closely. Persistence is therefore a strong baseline, and the network must be improved before the main comparison.

Four update strategies. As new data arrive, the network is updated in one of four ways:

1. *Ordinary updating*: keep training the same weights.
2. *Periodic retraining*: regularly reset all weights and retrain.
3. *Shrink-and-perturb* [2]: $\theta \leftarrow \lambda\theta + \sigma\varepsilon$, which shrinks the weights θ and adds fresh random noise ε.
4. *ReDo-style reset* [3]: re-initialise only the inactive neurons.

Fair, causal updates. Every few hours, each model trains on the most recent examples *whose future values have already been observed*, so no method sees the future. All strategies get the same examples and the same number of training steps. We log the computation each one uses.

6. Evaluation

Split. Test 1 trains the starting model. Test 2 selects all settings. Test 3 is used only once, after every setting is fixed. We have looked at summary statistics of Test 3, but have not scored any model on it.

Metrics. We report MAE per bearing and overall, MAE near failure, average underprediction, and computing cost (run time, number of updates and resets). Each method is run with five random seeds, and we report the differences between methods with their spread. The comparison targets are the baselines in Table 2 and the two reference strategies (ordinary updating and periodic

retraining). Ablations change one design choice at a time, such as how often models update.

Secondary study: can internal signals predict a helpful reset? At regular checkpoints we copy the ordinarily updated model and continue it in two ways: as is, or after a reset. We then measure which copy forecasts the next day better. Before each copy, we record two internal signals: the fraction of inactive neurons, and how varied the neurons' outputs are (effective rank [13]). We test whether these signals predict the benefit better than recent forecast error does.

If plasticity loss does not appear. We will first check on a small synthetic task that our code can detect plasticity loss at all. If the bearing data show little loss, we will report that as a result. The core deliverable is the forecasting comparison; a second bearing dataset is optional.

7. Resources and code

Everything runs on a laptop with Python, NumPy, pandas [14] and Matplotlib [15]. We expect tens of CPU-hours at most and do not need a GPU or AUB's HPC cluster. The repository contains the data download with checksum verification, the baselines, the neural network, saved results and setup instructions. All members commit regularly and disclose AI assistance.

Repository: `https://github.com/karimbteish/plasticity-bearing-forecasting`

8. Timeline and responsibilities

Elissa leads data, Galia leads models and update strategies, and Karim leads evaluation. All three write code and the report.

Week	Milestone (owner)
Oct 19	Proposal, data audit, baselines, repository (all).
Oct 26	Recompute features from NASA recordings (Elissa); improve the network (Galia).
Nov 2	Update strategies (Galia); logging and cost tracking (Karim).
Nov 9	Tune on Test 2 and fix the protocol (Galia, Karim); internal-signal pilot (Elissa, Karim).
Nov 16	Progress report, Nov 17 (all); five-seed results (Galia, Karim).
Nov 23	Test-3 evaluation and internal-signal analysis (Elissa, Karim).
Nov 30	Figures, poster and rehearsal; poster session Dec 4 (all).
Dec 7	Final report; submit Dec 10 (all).

References

[1] Shibhansh Dohare, J. Fernando Hernandez-Garcia, Qingfeng Lan, Parash Rahman, A. Rupam Mahmood, and Richard S. Sutton. Loss of plasticity in deep continual learning. *Nature*, 632:768–774, 2024. doi: 10.1038/s41586-024-07711-7.

[2] Jordan T. Ash and Ryan P. Adams. On warm-starting neural network training. In *Advances in Neural Information Processing Systems*, volume 33, 2020. URL https://proceedings.neurips.cc/paper/2020/hash/288cd2567953f06e460a33951f55daaf-Abstract.html.

[3] Ghada Sokar, Rishabh Agarwal, Pablo Samuel Castro, and Utku Evci. The dormant neuron phenomenon in deep reinforcement learning. In *Proceedings of ICML*, volume 202 of *Proceedings of Machine Learning Research*, pages 32145–32168, 2023. URL https://proceedings.mlr.press/v202/sokar23a.html.

[4] Clare Lyle, Zeyu Zheng, Evgenii Nikishin, Bernardo Avila Pires, Razvan Pascanu, and Will Dabney. Understanding plasticity in neural networks. In *Proceedings of ICML*, volume 202 of *Proceedings of Machine Learning Research*, pages 23190–23211, 2023. URL https://proceedings.mlr.press/v202/lyle23b.html.

[5] Jiuqi Wang, Jayanth Srinivasa, Claire Chen, Shuze Daniel Liu, Ali Payani, and Shangtong Zhang. Predicting plasticity in deep continual learning: A theoretical perspective. arXiv preprint 2605.09044, 2026. Version 1, 9 May 2026.

[6] Haonan Yang, Jianchao Tang, and Shaowu Yang. TFGI: a continual learning framework for remaining useful life prediction based on temporal fusion model. *The Journal of Supercomputing*, 82:529, 2026. doi: 10.1007/s11227-026-08667-3. Published 24 June 2026.

[7] J. Lee, H. Qiu, G. Yu, J. Lin, and Rexnord Technical Services. IMS bearing data set. NASA Ames Prognostics Data Repository; data from IMS, University of Cincinnati, 2007. URL https://data.nasa.gov/dataset/ims-bearings. Accessed 20 October 2026; catalog license: other-license-specified, linking U.S. government-work guidance.

[8] fatemeh-studio. nasa-bearing-anomaly: processed IMS feature tables and data documentation. GitHub repository, 2026. URL https://github.com/fatemeh-studio/nasa-bearing-anomaly/tree/f8a40f9ec0b47aac027e69f631c1deb79c7bef66. Accessed 20 October 2026. Commit f8a40f9. Code: MIT; data terms documented separately.

[9] Hai Qiu, Jay Lee, Jing Lin, and Gang Yu. Wavelet filter-based weak signature detection method and its application on rolling element bearing prognostics. *Journal of Sound and Vibration*, 289(4–5):1066–1090, 2006. doi: 10.1016/j.jsv.2005.03.007.

[10] Arthur E. Hoerl and Robert W. Kennard. Ridge regression: Biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970. doi: 10.1080/00401706.1970.10488634.

[11] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. Array programming with NumPy. *Nature*, 585:357–362, 2020. doi: 10.1038/s41586-020-2649-2.

[12] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. URL https://arxiv.org/abs/1412.6980.

[13] Olivier Roy and Martin Vetterli. The effective rank: A measure of effective dimensionality. In *European Signal Processing Conference (EUSIPCO)*, pages 606–610, 2007. doi: 10.5281/zenodo.40328.

[14] Wes McKinney. Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010. doi: 10.25080/Majora-92bf1922-00a.

[15] John D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi: 10.1109/MCSE.2007.55.